Embedded Firmware Essentials: Week II

Debugging

Goals

1. Better develop my AI assisted software engineering skills.
2. Utilize the size command to show memory layout of my app.
3. Become familiar with debugging techniques.
   a. LOGE
   b. esp_err_to_name()
   c. Gdb
4. Show use of errno to debug POSIX functions
5. Debug heap memory usage.
6. Use sleep modes.

AI Usage

I. I uploaded the lecture slides to ChatGPT and had it generate me a study guide in which it summarized each section, gave me a practical use case for each topic, and then created step-by-step guides for demoing each assignment problem.

II. For the “idf.py size” assignment, I asked ChatGPT to explain where the data displayed is being read from, both in my project workspace and in memory when necessary (for example, where my source code is stored in memory.)

III. Copilot was used to configure my project workspace when necessary.

   A. I found that both ChatGPT and Copilot failed to help me configure LittleFS by simply asking them too, I had to do some manual digging.

I. idf.py size

```
angel@Gabriel:~/blinky$ idf.py size
Creating ESP32S3 image...
Merged 2 ELF sections.
Successfully created ESP32S3 image.
Generated /home/angel/blinky/build/blinky.bin
[5/5] cd /home/angel/blinky/build && /home/angel/.esp...rtition-table.bin /home/angel/blinky/build/blinky.bin
blinky.bin binary size 0x28200 bytes. Smallest app partition is 0x100000 bytes. 0xd7e00 bytes (84%) free.
Running ninja in directory /home/angel/blinky/build
Executing "ninja size"...
[0/1] cd /home/angel/blinky/build && /usr/bin/cmake -D "IDF_SIZE_TOOL=/home/angel/.espressif/python_env/idf6.1_py3.10_env/bin/python;-m;esp_idf_size" -
D MAP_FILE=/home/angel/blinky/build/blinky.map -D OUTPUT_JSON= -P /home/angel/esp/master/esp-idf/tools/cmake/run_size_tool.cmake
                              Memory Type Usage Summary
```

Memory Type/Section	Used [bytes]	Used [%]	Remain [bytes]	Total [bytes]
Flash Code	65714			
.text	65714			
DIRAM	47335	13.85	294425	341760
.text	32483	9.5		
.data	12412	3.63		
.bss	2440	0.71		
Flash Data	37208			
.rodata	36952			
.appdesc	256			
IRAM	16384	100.0	0	16384
.text	15356	93.73		
.vectors	1028	6.27		
RTC SLOW	36	0.44	8156	8192
.force_slow	36	0.44		
RTC FAST	24	0.29	8168	8192
.rtc_reserved	24	0.29		

```
Total image size: 164237 bytes (.bin may be padded larger)
Note: The reported total sizes may be smaller than those in the technical reference manual due to reserved memory and application configuration. The to
tal flash size available for the application is not included by default, as it cannot be reliably determined due to the presence of other data like the
 bootloader, partition table, and application partition size.
```

I. idf.py size run on my blinking LED program

II. Logging

```
angel@Gabriel:~/logging-demo$ '/home/angel/.espressif/python_env/idf6.1_py3.10_env/bin/python' '/home/angel/esp/master/esp-idf/tools/idf_monitor.p
y' -p /dev/ttyACM0 -b 115200 --toolchain-prefix xtensa-esp32s3-elf- --make ''/home/angel/.espressif/python_env/idf6.1_py3.10_env/bin/python' '/hom
e/angel/esp/master/esp-idf/tools/idf.py'' --target esp32s3 '/home/angel/logging-demo/build/logging-demo.elf'
I (106) esp_image: segment 3: paddr=00020020 vaddr=42000020 size=0fb48h ( 64328) map
I (122) esp_image: segment 4: paddr=[illegible] 2976) load
I (129) esp_image: segment 5: paddr=[illegible] 36) load
I (135) boot: Loaded app from partit[illegible]
I (135) boot: Disabling RNG early entropy source...
I (147) cpu_start: Multicore app
I (156) cpu_start: GPIO 44 and 43 are used as console UART I/O pins
I (157) cpu_start: Pro cpu start user code
I (157) cpu_start: cpu freq: 160000000 Hz
I (158) app_init: Application information:
I (162) app_init: Project name:     logging-demo
I (166) app_init: App version:      1
I (170) app_init: Compile time:     Jan 31 2026 12:24:33
I (175) app_init: ELF file SHA256:  e07f2c1f9...
I (179) app_init: ESP-IDF:          v6.1-dev-2210-gf621cc4fff
I (184) efuse_init: Min chip rev:     v0.0
I (188) efuse_init: Max chip rev:     v0.99
I (192) efuse_init: Chip rev:         v0.2
I (196) heap_init: Initializing. RAM available for dynamic allocation:
I (202) heap_init: At 3FC93908 len 00055E08 (343 KiB): RAM
I (208) heap_init: At 3FCE9710 len 00005724 (21 KiB): RAM
I (213) heap_init: At 3FCF0000 len 00008000 (32 KiB): DRAM
I (218) heap_init: At 600FE000 len 00001FE8 (7 KiB): RTCRAM
I (224) spi_flash: detected chip: gd
I (226) spi_flash: flash io: dio
W (229) spi_flash: Detected size(8192k) larger than the size in the binary image header(2048k). Using the size in the binary image header.
I (242) sleep_gpio: Configure to isolate all GPIO pins in sleep state
I (248) sleep_gpio: Enable automatic switching of GPIO sleep configuration
I (255) main_task: Started on CPU0
I (265) main_task: Calling app_main()
I (265) LOG_DEMO: Starting logging demo
E (265) LOG_DEMO: Error in function app_main: ESP_FAIL
I (265) LOG_DEMO: Logging demo complete
```

Navigate to Command
Scroll to Previous Command (Ctrl+UpArrow)
Scroll to Next Command (Ctrl+DownArrow)

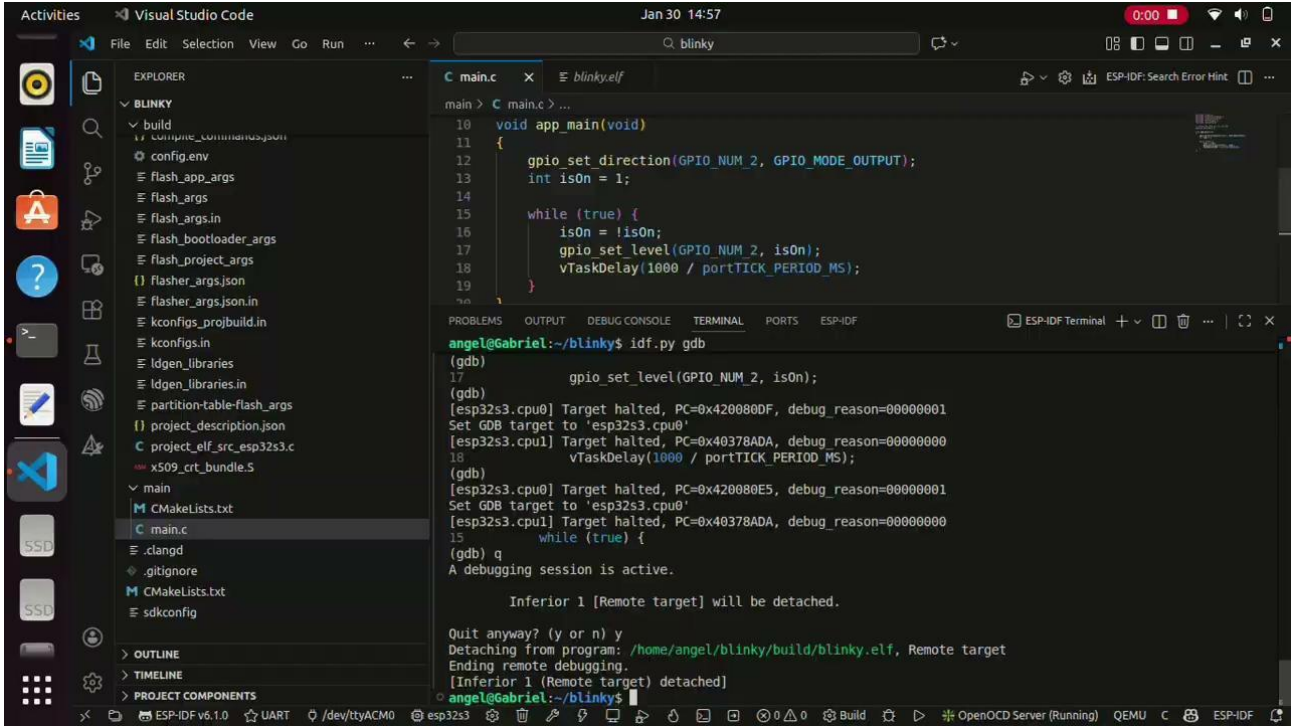
I. A generic error

II. Logging

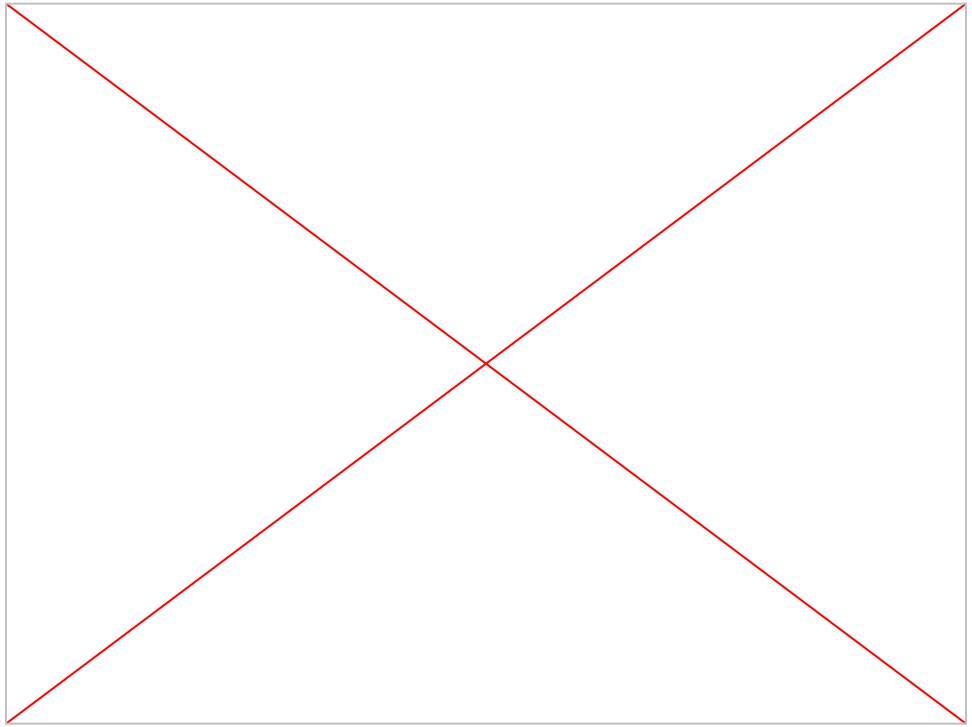
```
e/angel/esp/master/esp-idf/tools/idf.py'' --target esp32s3 '/home/angel/logging-demo/build/logging-demo.elf'
I (209) heap_init: At 3FCE9710 len 00005724 (21 KiB): RAM
I (214) heap_init: At 3FCF0000 len 00008000 (32 KiB): DRAM
I (220) heap_init: At 600FE000 len 00001FE8 (7 KiB): RTCRAM
I (226) spi_flash: detected chip: gd
I (228) spi_flash: flash io: dio
W (231) spi_flash: Detected size(8192k) larger than the size in the binary image header(2048k). Using the size in the binary image header.
I (243) sleep_gpio: Configure to isolate all GPIO pins in sleep state
I (250) sleep_gpio: Enable automatic switching of GPIO sleep configuration
I (257) main_task: Started on CPU0
I (267) main_task: Calling app_main()
I (267) LOG_DEMO: Starting real error logging demo
E (267) LOG_DEMO: Error in function app_main: ESP_ERR_NVS_NOT_INITIALIZED
I (277) LOG_DEMO: Demo complete
I (277) main_task: Returned from app_main()
```

I. An error thrown by trying to access an NVS system that has not been initialized.

II. GDB on screen



II. GDB IRL



III. Filesystem Mounting and Error Logging

```
I (263) main_task: Started on CPU0
I (283) main_task: Calling app_main()
I (283) FS_ERRNO: Mounting LittleFS...
E (283) esp_littlefs: partition "storage" could not be found
E (283) esp_littlefs: Failed to initialize LittleFS
E (293) FS_ERRNO: Failed to mount LittleFS: ESP_ERR_NOT_FOUND
I (293) main_task: Returned from app_main()
```

I. Triggering errors by failing to partition device for filesystem.

LittleFS Configuration steps

I. Download dependency
   A. idf.py add-dependency joltwallet/littlefs
II. Create partition table file (partitions.csv) in root directory.
III. Enable LittleFS through menuconfig
IV. Ensure ESPTOOL_FLASHSIZE is configured to be large enough for your partition table.
V. Proceed to writing code!

III. Filesystem Mounting and Error Logging

```
I (267) sleep_gpio: Configure to isolate all GPIO pins in sleep state
I (273) sleep_gpio: Enable automatic switching of GPIO sleep configuration
I (280) main_task: Started on CPU0
I (300) main_task: Calling app_main()
I (300) FS_ERRNO: Mounting LittleFS...
I (300) FS_ERRNO: LittleFS mounted
E (300) FS_ERRNO: fopen failed: errno=2 (No such file or directory)
I (310) main_task: Returned from app_main()
```

I. Successfully mounting LittleFS filesystem

IV. Debugging Heap

```
(2048k). Using the size in the binary image header.
I (242) sleep_gpio: Configure to isolate all GPIO pins in sleep state
I (248) sleep_gpio: Enable automatic switching of GPIO sleep configuration
I (255) main_task: Started on CPU0
I (265) main_task: Calling app_main()
I (265) HEAP: Heap at startup
I (265) HEAP:   Free heap: 398684 bytes
I (265) HEAP:   Minimum ever free heap: 398684 bytes
I (275) HEAP: Allocating 1024 bytes...
I (275) HEAP: Heap after malloc
I (275) HEAP:   Free heap: 397656 bytes
I (285) HEAP:   Minimum ever free heap: 397656 bytes
I (285) HEAP: Freeing memory...
I (285) HEAP: Heap after free
I (295) HEAP:   Free heap: 398684 bytes
I (295) HEAP:   Minimum ever free heap: 397656 bytes
I (305) main_task: Returned from app_main()
```

I. Allocating and then freeing memory on the heap.

IV. Debugging Heap

```
I (224) spi_flash: detected chip: gd
I (227) spi_flash: flash io: dio
W (230) spi_flash: Detected size(8192k) larger than the size in the binary image header
(2048k). Using the size in the binary image header.
I (242) sleep_gpio: Configure to isolate all GPIO pins in sleep state
I (248) sleep_gpio: Enable automatic switching of GPIO sleep configuration
I (255) main_task: Started on CPU0
I (265) main_task: Calling app_main()
I (265) HEAP: Heap at startup
I (265) HEAP:   Free heap: 398684 bytes
I (265) HEAP:   Minimum ever free heap: 398684 bytes
I (275) HEAP: Allocating 1024 bytes (leaked)...
I (275) HEAP: Heap after leaking memory
I (275) HEAP:   Free heap: 397656 bytes
I (285) HEAP:   Minimum ever free heap: 397656 bytes
I (285) main_task: Returned from app_main()
```

I. Allocating and leaking memory.

V. Sleep Modes

```
port?)
--- Waiting for the device to reconnect..........
I (2326) SLEEP: Entering light sleep for 5 seconds...
I (2326) SLEEP: Woke up from light sleep
I (4326) SLEEP: Entering deep sleep for 5 seconds...
I (4326) SLEEP: USB serial will disconnect now
--- Error: device reports readiness to read but returned no data (device disconnected or multiple access on
port?)
--- Waiting for the device to reconnect..........
I (2321) SLEEP: Entering light sleep for 5 seconds...
I (2321) SLEEP: Woke up from light sleep
I (4321) SLEEP: Entering deep sleep for 5 seconds...
I (4321) SLEEP: USB serial will disconnect now
--- Error: device reports readiness to read but returned no data (device disconnected or multiple access on
port?)
--- Waiting for the device to reconnect....
```

I. Demoing sleep modes